

Code of the Day

- Asynchronous GPU memory allocation using `cudaMallocAsync`
 - Recall `cudaMalloc` is blocking

```
// Create a stream
cudaStreamCreate(&stream);

// Use cudaMallocAsync to allocate memory asynchronously in the stream
cudaMallocAsync(&d_data, N * sizeof(int), stream);

// Initialize host data - overlap with cudaMallocAsync
int *h_data = (int *)malloc(N * sizeof(int));
for (int i = 0; i < N; i++) {
    h_data[i] = i;
}

// Copy data to device asynchronously
cudaMemcpyAsync(d_data, h_data, N*sizeof(int), cudaMemcpyHostToDevice, stream);
```

The two operations can overlap!

ECE 759

High Performance Computing for Applications in Engineering

[Spring 2025]

Scan

04/18/2025

Before we get started...

- Quick overview, things discussed last time
 - GPU-parallel reduction kernel and optimization techniques
 - Global memory, shared memory, bank conflict, warp unrolling
- Purpose of today's lecture: GPU-parallel scan kernel
- Miscellaneous
 - Programming assignment #7 is released – due **Friday 5/2 at 23:50 PM**
 - Take-home final exams will take place during the week of 4/25 – 5/4
 - In-class final project presentation take place on 4/25, 4/28, 5/2 – Please sign up [here](#)
 - You will need to upload your slides by noon on your scheduled presentation day
 - Link will be available on Canvas
 - Please complete course evaluation: <https://it.wisc.edu/services/heliocampusac/>
 - Bonus question in final exam: how many questions did you answer?

What is a Scan Operation?

- The scan operation takes a binary associative operator $\oplus$ with an array of n elements

$$[a_0, a_1, \dots, a_{n-1}]$$

and returns the ordered set

$$[a_0, (a_0 \oplus a_1), (a_0 \oplus a_1 \oplus a_2), \dots, (a_0 \oplus a_1 \oplus \dots \oplus a_{n-2}), (a_0 \oplus a_1 \oplus \dots \oplus a_{n-2} \oplus a_{n-1})].$$

- Example: If $\oplus$ is addition, then scan on the set

$$[3 \ 1 \ 7 \ 0 \ 4 \ 1 \ 6 \ 3]$$

returns the set

$$[3 \ 4 \ 11 \ 11 \ 15 \ 16 \ 22 \ 25]$$

- Scan represents the “sum” of all previous values, thus also known as *prefix sum* or *prefix scan*

Scan Operation Comes with Two Flavors

- **#1: exclusive scan** – last element does not participate in the operation

[3 1 7 0 4 1 6 3]

returns the set

[0 3 4 11 11 15 16 22]

- **#2: inclusive scan** – last element participates in the operation (i.e., all elements are involved)

[3 1 7 0 4 1 6 3]

returns the set

[3 4 11 11 15 16 22 25]

- They are equally useful; moreover, if one is available, the other one can be quickly obtained
 - You can achieve exclusive scan with inclusive scan by adding a dummy zero as the first element

Scan on the CPU is Very Easy to Implement

```
void exclusive_scan( float* scanned, float* input, int length)
{
    scanned[0] = 0;
    for(int i = 1; i < length; ++i)
    {
        scanned[i] = input[i-1] + scanned[i-1];
    }
}
```

- Just need a for-loop to iterate the given input array
 - Add each element to the sum of the elements before it
 - Repeat $n - 1$ such addition iterations – $O(n)$ time and work complexity
- C++ STL provides you with both scan operations
 - [`std::exclusive\_scan`](#)
 - [`std::inclusive\_scan`](#)

```
#include <functional>
#include <iostream>
#include <iterator>
#include <numeric>
#include <vector>

int main()
{
    std::vector data{3, 1, 4, 1, 5, 9, 2, 6};

    std::cout << "Exclusive sum: ";
    std::exclusive_scan(data.begin(), data.end(),
        std::ostream_iterator<int>(std::cout, " "),
        0);

    std::cout << "\nInclusive sum: ";
    std::inclusive_scan(data.begin(), data.end(),
        std::ostream_iterator<int>(std::cout, " "));

    std::cout << "\n\nExclusive product: ";
    std::exclusive_scan(data.begin(), data.end(),
        std::ostream_iterator<int>(std::cout, " "),
        1, std::multiplies<>{});

    std::cout << "\n\nInclusive product: ";
    std::inclusive_scan(data.begin(), data.end(),
        std::ostream_iterator<int>(std::cout, " "),
        std::multiplies<>{});
}
```

Prefix Scan is a Fundamental Building Block of Many Algorithms

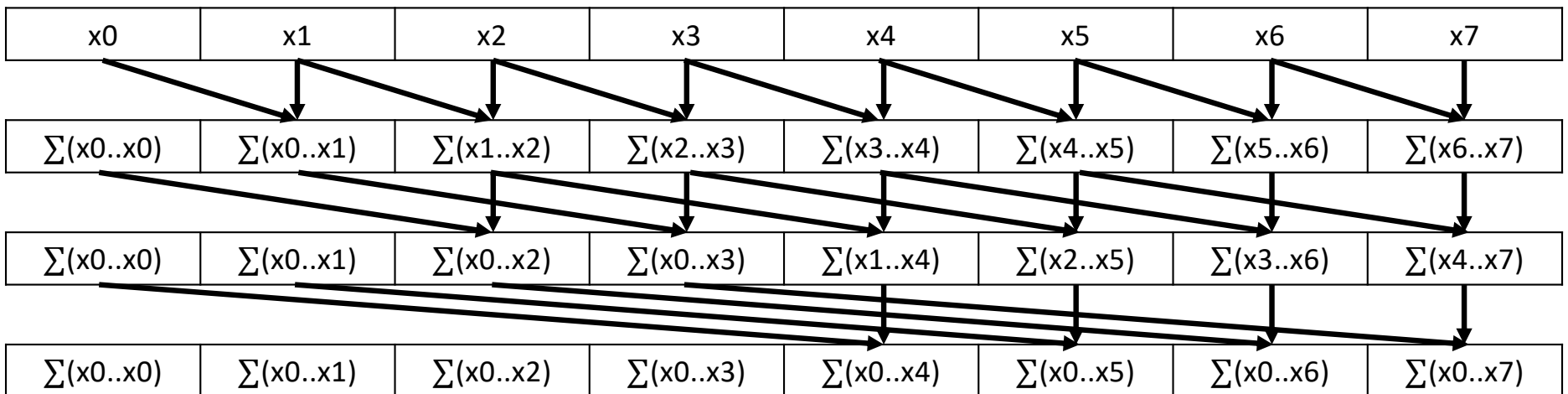
- Radix sort
- Quicksort
- String comparison
- Lexical analysis
- Stream compaction
- Polynomial evaluation
- Solving recurrences
- Tree operations
- Histograms
- Etc.

We Will Present Two GPU-parallel Scan Algorithms

- Scan algorithm #1: Hillis & Steele (1986)
 - Simple but not work-efficient algorithm
 - Note: An algorithm is work-inefficient if it consumes more work asymptotically than the best case of a sequential algorithm – in our case, CPU-based scan has a work complexity of $O(N)$
- Scan algorithm #2: Harris-Sengupta-Owen (2007)
 - Not so simple but work-efficient algorithm
- Both algorithms share one thing in common
 - They all resemble the idea of *reduction tree* that iteratively reduce two elements with a reduction distance doubling at each iteration
 - 1st iteration has a distance of 1, 2nd iteration has a distance of 2, 3rd iteration has a distance of 4, etc.
 - During the reduction iterations, the last entry of the array *anchors* the algorithm
 - The last entry of the array is modified at each iteration of the algorithm while the other elements are not updated so frequently

Parallel Scan Algorithm #1: Hillis & Steele (1986)

- Each thread iteratively combines and reduces two values from the array
 - The two values are separated by a distance that increases by a power of 2 – $\sum(x_i..x(i+2^d))$
- To accommodate partial results, the algorithm requires **two buffers** of length N
 - One buffer stores partial result and another buffer stores the solution from the previous iteration



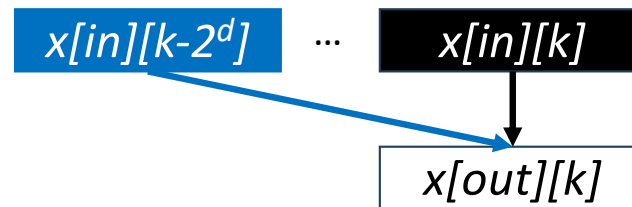
In our example, $n = 8 = 2^3$, the algorithm takes $M = 3$ parallel iterations ($d=0, 1, 2$)

General Idea of Hillis & Steele Algorithm

- First iteration ($d=0$), I go with stride $1=2^0$ to obtain prefix-sum of 2 elements
 - Here, the stride size represents the reduction distance between two elements
 - With $d=0$, start at $x[2^d-1]$ and apply this stride to all the array elements before $x[2^d-1]$ to find the pair of each of them. When looking for the pair, the stride should not land you before the beginning of the array. The sum replaces the element of higher index.
- Second iteration ($d=1$), I go with stride $2=2^1$ to obtain prefix-sum of 4 elements
 - With $d=1$, start at $x[2^d-1]$ and apply this stride to all the array elements before $x[2^d-1]$ to find the pair of each of them. When looking for the pair, the stride should not land you before the beginning of the array. The sum replaces the element of higher index.
- Third iteration ($d=2$): I go with stride $4=2^2$ to obtain prefix-sum of 8 elements
 - With $d=2$, start at $x[2^d-1]$ and apply this stride to all the array elements before $x[2^d-1]$ to find the pair of each of them. When looking for the pair, the stride should not land you before the beginning of the array. The sum replaces the element of higher index.
- ... (and so on)

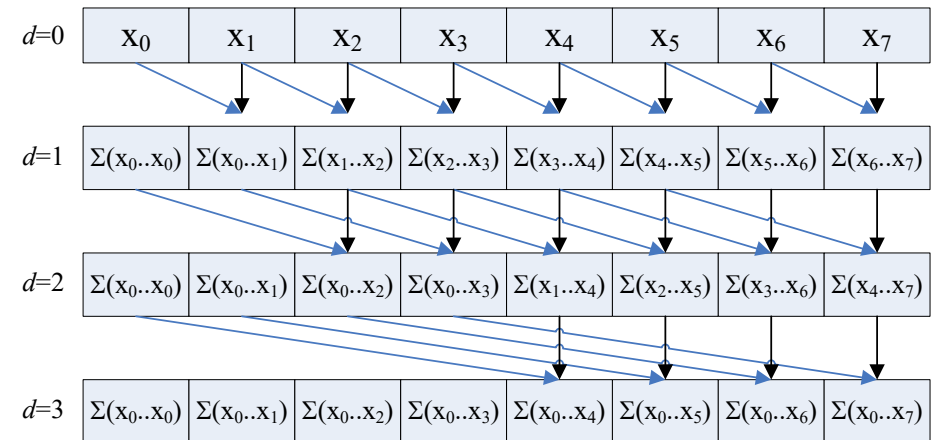
Pseudo Code of Hillis & Steele Algorithm

Two buffers $x[in]$ and $x[out]$ are needed to implement the algorithm



```

for  $d := 0$  to  $M-1$  do
  forall  $k$  do in parallel
    if  $k - 2^d \geq 0$  then
       $x[out][k] := x[in][k] + x[in][k - 2^d]$ 
    else
       $x[out][k] := x[in][k]$ 
    endforall
  swap( $in, out$ )
endfor
    
```



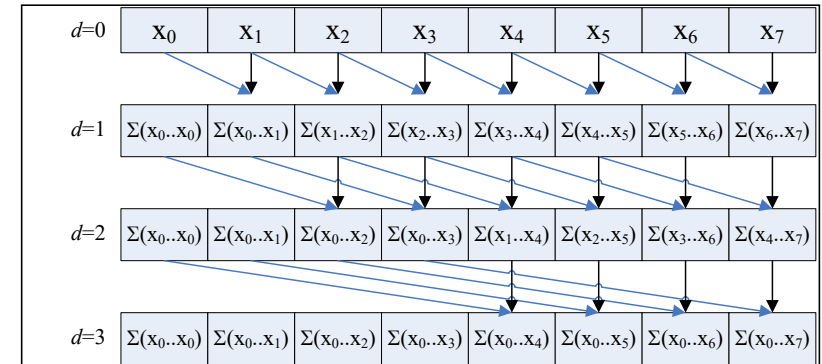
Hillis & Steele: Kernel Function

```
// Assume the input array fits into a single block for simplicity
__global__ void scan(float *g_odata, float *g_idata, int n) {
    extern volatile __shared__ float temp[]; // allocated on invocation
    int thid = threadIdx.x;
    int pout = 0, pin = 1;
    // load input into shared memory.
    temp[thid] = g_idata[thid];
    __syncthreads();

    for( int offset = 1; offset < n; offset *= 2 ) {
        pout = 1 - pout;           // swap double buffer indices
        pin  = 1 - pout;
        if (thid - offset >= 0) { // the left neighbor does not go beyond 0
            temp[pout*n+thid] = temp[pin*n+thid] + temp[pin*n + thid - offset];
        }
        else {
            temp[pout*n+thid] = temp[pin*n+thid];
        }
        __syncthreads(); // I need this here before I start next iteration
    }
    g_odata[thid] = temp[pout*n+thid]; // write output
}
```

Takeaway of Hillis & Steele Kernel

- The kernel is very simple to implement
- Time complexity: $M = O(\log_2 N)$ iterations needed
- Work complexity:



$M = \log_2(N)$ iterations needed

The number of operations: $(N-1) + (N-2) + (N-4) \dots + (N-2^{k-1}) + \dots + (N-2^{M-1})$

Final operation count: $(\log_2 N) * N - (2^0 + 2^1 + \dots + 2^{M-1}) = N \log_2 N - (n - 1)$

$$\sum_{i=1}^n a_1 r^{i-1} = S_n = \frac{a_1(1 - r^n)}{1 - r}, \quad r \neq 1$$

This is an algorithm with $O(N * \log(N))$ work – $\log(N)$ times more work than sequential implementation!

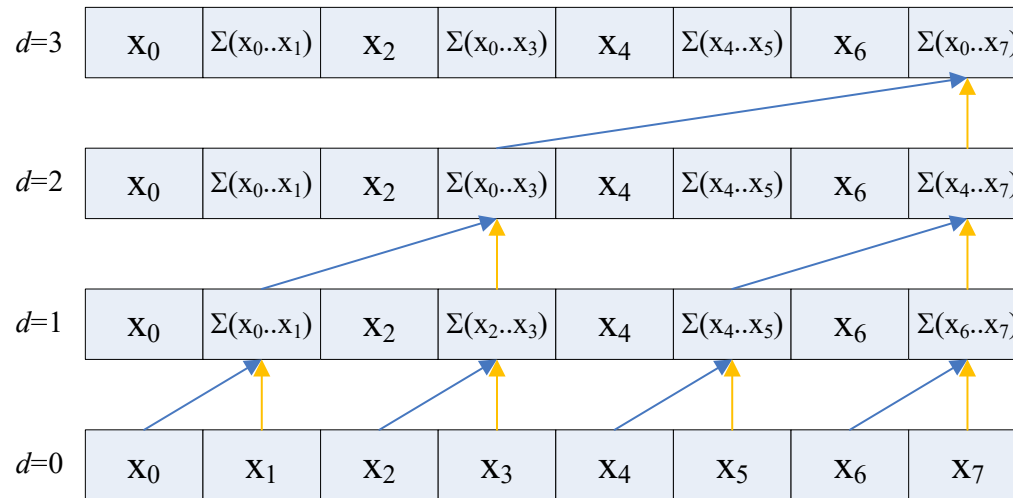
Ex: This $\log(N)$ factor hurts the performance and can incur 20x more work for 10^6 elements!

Parallel Scan Algorithm #2: Harris-Sengupta-Owen (2007)

- Leveraged the *balanced tree* idea to achieve the optimal work efficiency
 - Build a balanced binary tree on input data from leaves to root, and then back from root to leaves
 - Bottom-up sweep and top-down sweep to calculate partial sums
 - Tree is not actually built but an abstraction to help determine what each thread does at each iteration
- For balanced tree-based scan: there are two stages
 - Stage 1: Traverse **from leaves to root** building partial sums at internal nodes in the tree
 - Bottom-up sweep – basically a reduction tree
 - Stage 2: Traverse **from root to leaves** propagating partial results of stage 1 and reducing them to final sums
 - Top-down sweep – propagate reduced partial values to other entries
- Paper available [here](#) (+800 citations)

Stage #1: The Reduction Tree

Key idea: Iteratively reduce two elements with a distance of 2^d



NOTE: "-1" entries indicate no-ops

$j \cdot 2^{d+1} - 1 =$				
1	3	5	7	1 st step, $d=0 \rightarrow d=1$
3	7	-1	-1	2 nd step, $d=1 \rightarrow d=2$
7	-1	-1	-1	3 rd step, $d=2 \rightarrow d=3$

$j \cdot 2^{d+1} - 2^d - 1 =$				
0	2	4	6	1 st step, $d=0 \rightarrow d=1$
1	5	-1	-1	2 nd step, $d=1 \rightarrow d=2$
3	-1	-1	-1	3 rd step, $d=2 \rightarrow d=3$

```

for d=0 to M-1
  offset =  $2^d$ 
  for j=1 to  $2^{M-d-1}$  do in parallel
     $x[j \cdot 2^{d+1} - 1] = x[j \cdot 2^{d+1} - 1] + x[j \cdot 2^{d+1} - 2^d - 1]$ 
  endfor
endfor

```

Operation Count

$$M = \log_2 N$$

```
for d=0 to M-1
  offset = 2k
  for j=1 to 2M-d-1 do in parallel
    x[j·2d+1-1] = x[j·2d+1-1] + x[j·2d+1-2d-1]
  endfor
endfor
```

By inspection:

$$\sum_{d=0}^{M-1} 2^{M-d-1} = 2^M - 1 = n - 1$$

Work efficiency is the same as parallel reduction (last lecture): $O(n)$

$$\sum_{i=1}^n a_1 r^{i-1} = S_n = \frac{a_1(1 - r^n)}{1 - r}, \quad r \neq 1$$

Output of First Stage is Not a Complete Prefix Sum Solution ...

- The output of the 1st stage is a footprint of the reduction tree with a bunch of partial sums

x_0	$\sum(x_0..x_1)$	x_2	$\sum(x_0..x_3)$	x_4	$\sum(x_4..x_5)$	x_6	$\sum(x_0..x_7)$
-------	------------------	-------	------------------	-------	------------------	-------	------------------

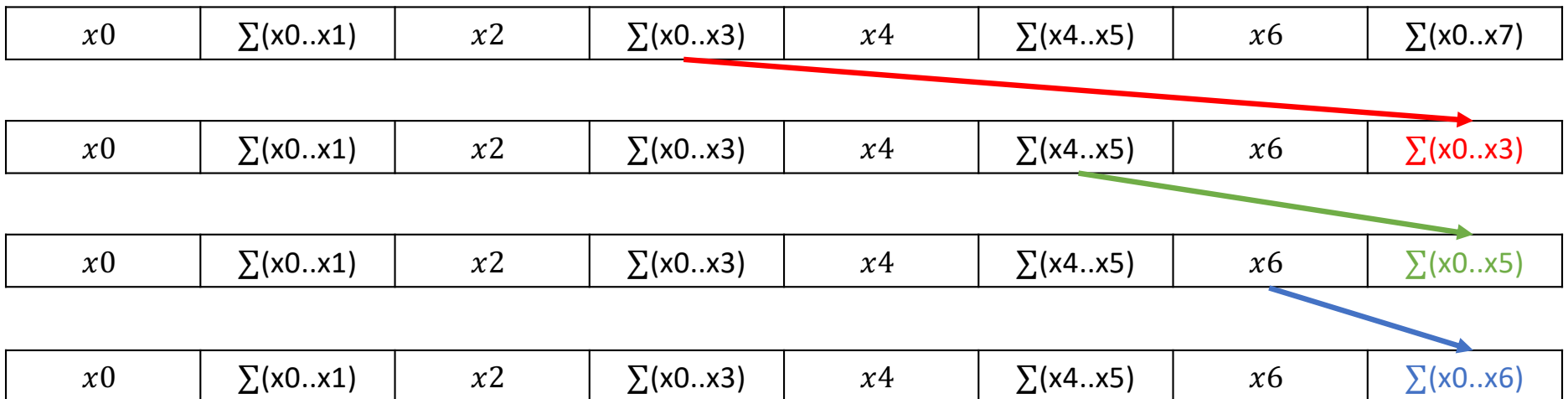


However, we want the exclusive-scan result below:

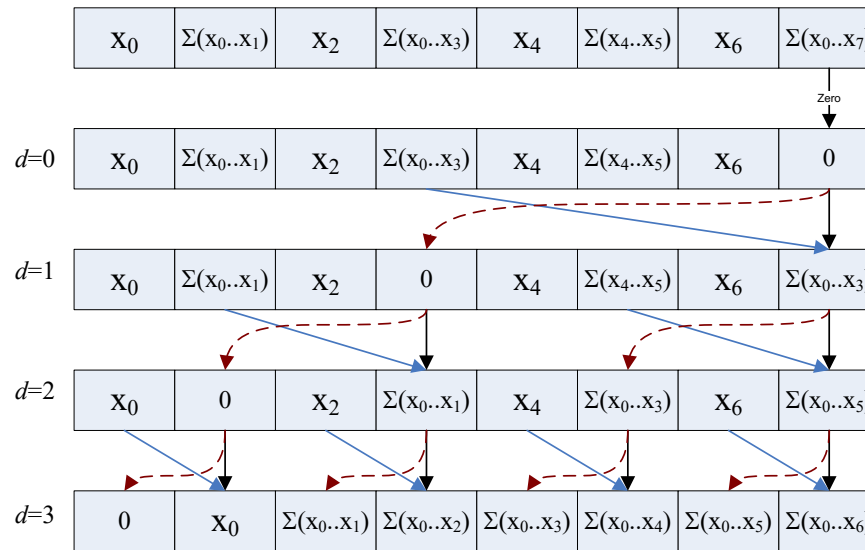
0	x_0	$\sum(x_0..x_1)$	$\sum(x_0..x_2)$	$\sum(x_0..x_3)$	$\sum(x_0..x_4)$	$\sum(x_0..x_5)$	$\sum(x_0..x_6)$
---	-------	------------------	------------------	------------------	------------------	------------------	------------------

How to Obtain the Prefix Sum of $\sum(x0..x6)$?

- Our reduction tree has three levels $\rightarrow$ we have three reduction distances 1, 2, and 4
 - How do we leverage the three distance values to find the prefix sum of each element?
 - Ex: In exclusive scan, the last element should be $\sum(x0..x6)$



Stage #2: The Top-Down Sweep



NOTE: This is just a mirror image of the reduction stage. Tricky to come up with the indexing scheme...

```

for k=M-1 to 0
  offset = 2k
  for j=1 to 2M-k-1 do in parallel
    temp = x[j·2k+1-2k-1]
    x[j·2k+1-2k-1] = x[j·2k+1-1]
    x[j·2k+1-1] = x[j·2k+1-1] + temp
  endfor
endfor
    
```

Annotations for the code block:

- Obtain the partial sum:** Points to `temp = x[j·2k+1-2k-1]`
- Propagate the partial sum:** Points to `x[j·2k+1-2k-1] = x[j·2k+1-1]`
- Reduce:** Points to `x[j·2k+1-1] = x[j·2k+1-1] + temp`

Complexity of Harris-Sengupta-Owen Scan Algorithm

- The implementation assume a single-block scan
 - Multiple-block scan can be done similarly to the reduction iteration we presented before
- Both stages are nothing but a reduction tree
 - The number of operations needed forms a geometric series with $\log N$ terms (= # of reduction levels)
 - The number of operations needed is $1 + 2 + 4 + 8 + \dots + \log N = O(N)$
- Specifically, the total number of operations associated with this algorithm
 - Additions: $2n-2$ ($n-1$ additions for each stage)
 - Overwrite ops: $n-1$ (needed only by the down-sweep stage)
 - Comparable with the operation count in the sequential solution – the algorithm is *work-efficient*

```
__global__ void prescan(float *g_odata, float *g_idata, int n){
    extern volatile __shared__ float temp[]; // allocated on invocation
    int thid = threadIdx.x;
    int offset = 1;
    temp[2 * thid] = g_idata[2 * thid]; // load input into shared memory
    temp[2 * thid + 1] = g_idata[2 * thid + 1];
    for (int d=n >> 1; d > 0; d >>= 1){ // build sum in place up the tree
        __syncthreads();
        if (thid < d) {
            int ai = offset * (2 * thid + 1) - 1;
            int bi = offset * (2 * thid + 2) - 1;
            temp[bi] += temp[ai];
        }
        offset <<= 1; // multiply by 2 implemented as bitwise operation
    }
    if (thid == 0) {
        temp[n - 1] = 0;
    } // clear the last element
    for (int d = 1; d < n; d *= 2){ // traverse down tree & build scan
        offset >>= 1;
        __syncthreads();
        if (thid < d){
            int ai = offset * (2 * thid + 1) - 1;
            int bi = offset * (2 * thid + 2) - 1;
            float t = temp[ai];
            temp[ai] = temp[bi];
            temp[bi] += t;
        }
    }
    __syncthreads();
    g_odata[2 * thid] = temp[2 * thid]; // write results to device memory
    g_odata[2 * thid + 1] = temp[2 * thid + 1];
}
```

Performance Comparison between CPU- and GPU-parallel Scans

- In general, the speed-up of GPU over CPU increases as the number of elements increases

# elements	CPU Scan (ms)	GPU Scan (ms)	Speedup
1024	0.002231	0.079492	0.03
32768	0.072663	0.106159	0.68
65536	0.146326	0.137006	1.07
131072	0.726429	0.200257	3.63
262144	1.454742	0.326900	4.45
524288	2.911067	0.624104	4.66
1048576	5.900097	1.118091	5.28
2097152	11.848376	2.099666	5.64
4194304	23.835931	4.062923	5.87
8388688	47.390906	7.987311	5.93
16777216	94.794598	15.854781	5.98

Summary of Parallel Scan Algorithms

- Sequential scan is easy to implement
 - Time complexity: $O(N)$
 - Work complexity: $O(N)$
- Hillis-Steele algorithm is simple but not work-efficient
 - Time complexity: $O(\log N)$
 - Work complexity: $O(N \log N)$
- Harris-Sengupta-Owen algorithm is quite complicated in indexing but work-efficient
 - Time complexity: $O(\log N)$
 - Work complexity: $O(N)$
- In my experience, modern CPUs are quite fast – you need N up to $4M$ to benefit from GPU